

Vulnerability Report: SQL Injection and Authentication Bypass in Vuln-Bank

Prepared for: MuLearn Bootcamp

1. Introduction

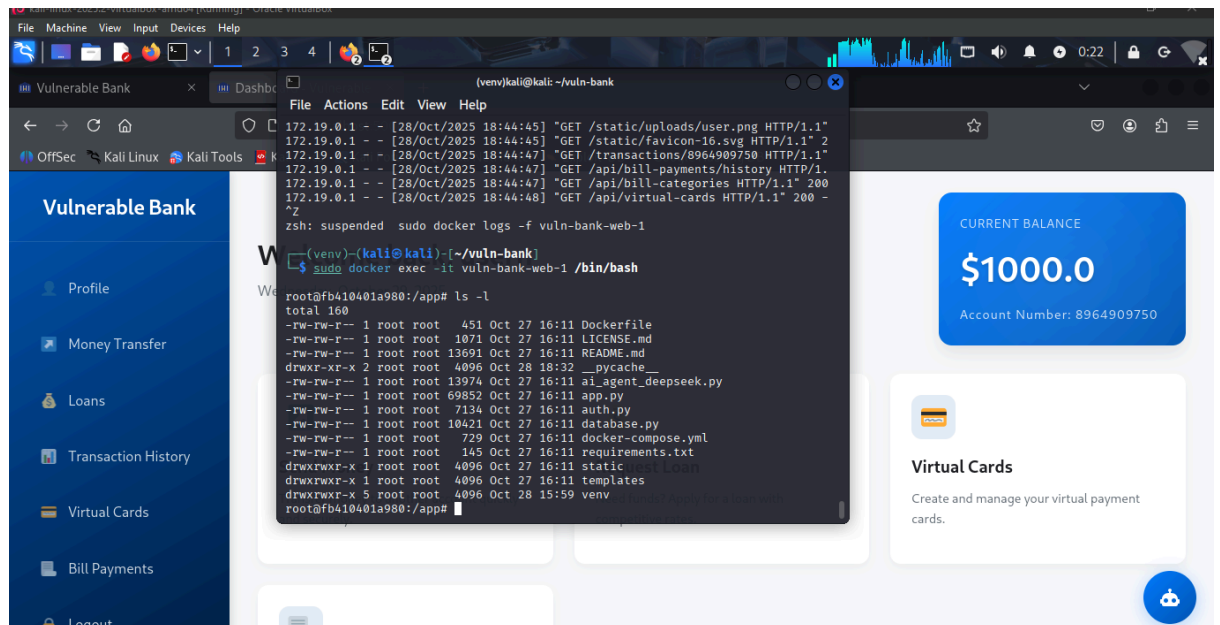
This report documents a beginner-level penetration test of the purposely vulnerable banking web application, Vuln-Bank. The goal was to identify critical flaws in the authentication system, capture original evidence.

2. Executive Summary

- Vuln-Bank was tested for SQL Injection, authentication bypass, and privilege escalation issues.
 - A high-risk flaw was identified: the login code allows direct injection of user input into SQL queries.
 - Authentication and admin privileges can be bypassed with carefully crafted payloads.
-

3. Lab Setup

- Vuln-Bank repo cloned from GitHub <https://github.com/Commando-X/vuln-bank.git>
- Docker containers deployed on Kali Linux VM .



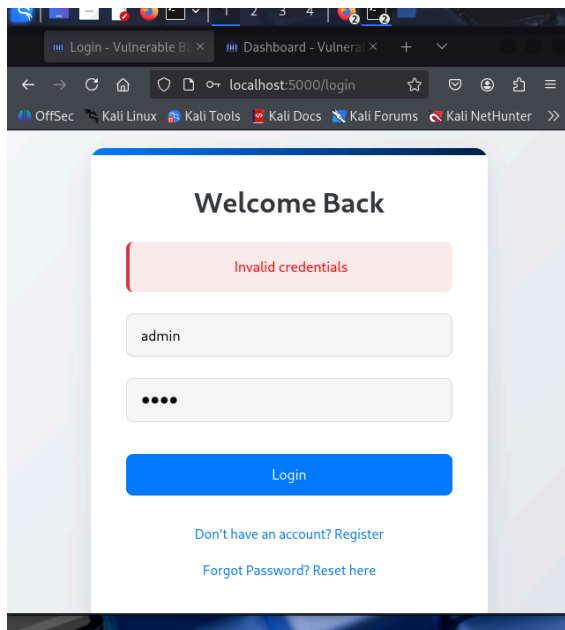
4. Methodology

- Git, Docker, and Docker Compose installed for environment setup.
 - Source code reviewed; logging statements added for SQL queries and input.
 - Manual and automated login attempts performed, using normal credentials and SQL injection payloads.
 - UI navigation and server logs reviewed for every critical action ..
-

5. Security Assessment Findings

5.1 Normal Login

- Logging in with expected credentials (admin/admin123 or test/test123) returns the correct dashboard or error state.
- No known vulnerabilities exploited in this step.



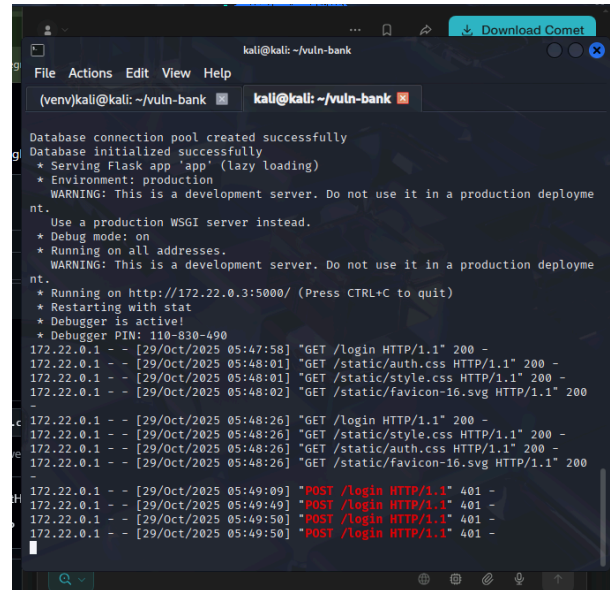
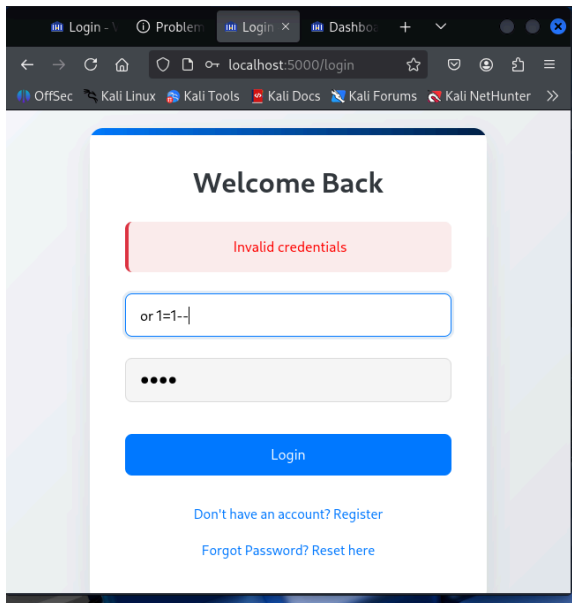
```
kali@kali: ~/vuln-bank
File Actions Edit View Help
(venv)kali@kali: ~/vuln-bank  kali@kali: ~/vuln-bank

Database connection pool created successfully
Database initialized successfully
* Serving Flask app 'app' (lazy loading)
* Environment: production
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Debug mode: on
* Running on all addresses.
WARNING: This is a development server. Do not use it in a production deployment.
* Restarting with stat
* Debugger is active!
* Debugger PIN: 110-830-490
172.22.0.1 - - [29/Oct/2025 05:47:58] "GET /login HTTP/1.1" 200 -
172.22.0.1 - - [29/Oct/2025 05:48:01] "GET /static/auth.css HTTP/1.1" 200 -
172.22.0.1 - - [29/Oct/2025 05:48:01] "GET /static/style.css HTTP/1.1" 200 -
172.22.0.1 - - [29/Oct/2025 05:48:02] "GET /static/favicon-16.svg HTTP/1.1" 200 -
172.22.0.1 - - [29/Oct/2025 05:48:26] "GET /login HTTP/1.1" 200 -
172.22.0.1 - - [29/Oct/2025 05:48:26] "GET /static/style.css HTTP/1.1" 200 -
172.22.0.1 - - [29/Oct/2025 05:48:26] "GET /static/auth.css HTTP/1.1" 200 -
172.22.0.1 - - [29/Oct/2025 05:48:26] "GET /static/favicon-16.svg HTTP/1.1" 200 -
172.22.0.1 - - [29/Oct/2025 05:49:09] "POST /login HTTP/1.1" 401 -
172.22.0.1 - - [29/Oct/2025 05:49:49] "POST /login HTTP/1.1" 401 -
172.22.0.1 - - [29/Oct/2025 05:49:50] "POST /login HTTP/1.1" 401 -
172.22.0.1 - - [29/Oct/2025 05:49:50] "POST /login HTTP/1.1" 401 -
```

5.2 SQL Injection Test

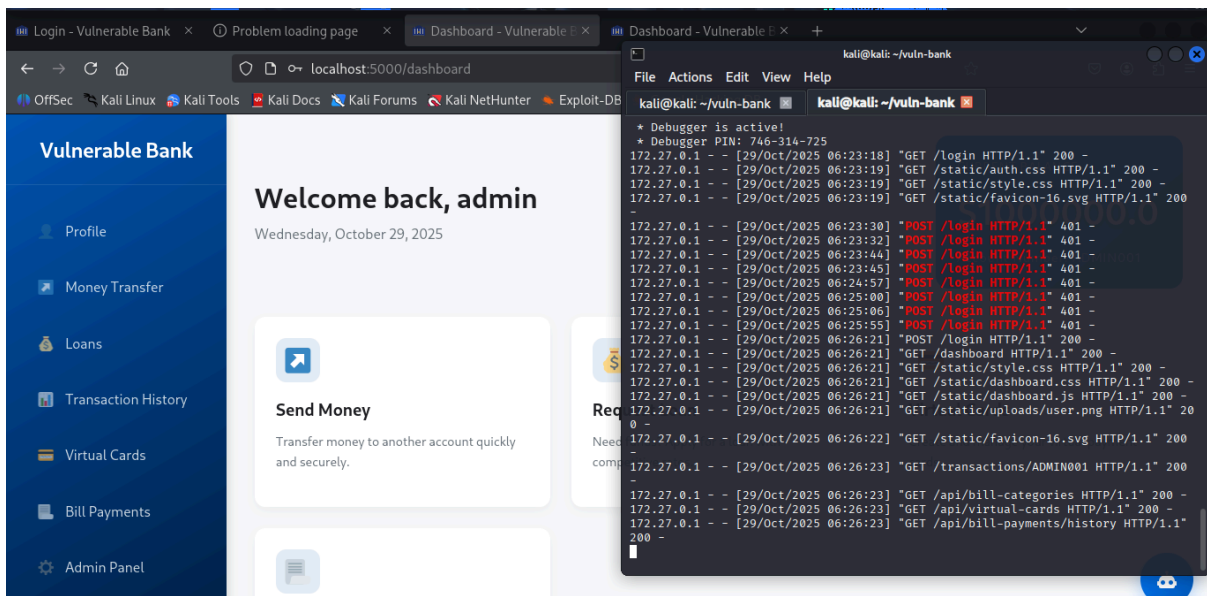
- Payloads like ' OR '1'='1 and or 1=1-- were injected in the username field.
- SQL query debug output confirmed that user input is directly embedded in the SQL statement.

Authentication bypass succeeded, allowing unauthorized dashboard/admin panel access.



5.3 Admin Functionality

- After bypass, the tester accessed admin panel and performed privileged actions (viewed/approved loans, edited users).
- Successful privilege escalation demonstrated in the dashboard .



6. Remediation Advice

- Always use parameterized queries—NEVER concatenate user data into SQL statements.
- Sanitize all input fields to prevent injection.
- Use secure password storage (hash + salt).
- Restrict access controls carefully; implement MFA for admin accounts.
- Restrict logging of sensitive input data (especially passwords and user payloads).
- Add proper error handling and suppress raw server errors in production.

Step 6: Terminate the Application

- To shut down vuln-bank, return to the project directory in your terminal and run:

`docker-compose down`
- This safely terminates all active containers.

Conclusion

The vulnerability assessment of the vuln-bank web application revealed key security deficiencies:

- **Authentication flaws:** SQL injection enables bypassing standard login security checks.
- **Data protection:** Weak password management makes reset endpoints more vulnerable.
- **Transaction processing weaknesses:** Unauthorized actions may affect financial records and user privacy.

Recommended improvements include input validation, better authentication mechanisms, secure session management, and regular updates to safeguard against future threats.

References

This assessment leveraged official vuln-bank documentation and hands-on observations.
Cite: [Commando-X vuln-bank README], your own screenshots, and code samples.
